

Logos Experiment 5 Report

Weight Tying + SwiGLU Feed-Forward Network

1. Experiment Objective

This experiment combines the two most useful findings from earlier Logos runs. Weight tying reduced the parameter count and checkpoint size, while the SwiGLU feed-forward network improved validation quality. The goal of Experiment 5 was to test whether these two improvements can be combined into one model that is both smaller and more capable.

2. Research Question

Can a decoder-only Transformer using both weight tying and a SwiGLU feed-forward network achieve better validation loss and perplexity than the original baseline while using far fewer parameters and a smaller checkpoint?

Hypothesis

Combining weight tying with SwiGLU should preserve the storage and parameter-efficiency benefits of weight tying while improving validation loss and perplexity through the stronger gated feed-forward block.

3. Method Summary

Component	Experiment 5 Setting
Architecture	Decoder-only GPT-style causal language model
Tokenizer	GPT-2 tokenizer
Dataset	OpenWebText subset, 10,000 text samples
Task	Next-token prediction
Normalization	LayerNorm
Feed-forward block	SwiGLU
Weight tying	Enabled: token embedding weight shared with output LM head
Context length	256 tokens
Training budget	1500 iterations

4. Configuration

Setting	Value
Run name	logos_exp05_weight_tying_swiglu
Batch size	64
Block size	256
Embedding dimension	256
Attention heads	8
Transformer layers	8
Dropout	0.2
SwiGLU hidden size	682
Learning rate	3e-4
Optimizer	AdamW
Weight decay	0.1
Gradient clipping	1.0
Mixed precision	Enabled on CUDA

5. Final Results

Metric	Experiment 5 Result
Parameters	19,242,400
FFN type	SwiGLU
SwiGLU hidden size	682
Weight tying active	True
Best iteration	1350
Best validation loss	5.7230
Final validation loss	5.7247
Final validation perplexity	306.36
Checkpoint size	73.44 MB
Peak VRAM	11.18 GB
Training time	157.07 sec, about 2.62 min
Training speed	about 157,166 tokens/sec
Generation speed	66.47 tokens/sec
Generation latency	15.04 ms/token
Average GPU power	243.58 W
Estimated GPU energy	10.49 Wh

6. Comparison with Key Previous Runs

Metric	Baseline	Weight Tying	SwiGLU	WT + SwiGLU
Parameters	32.16M	19.24M	32.16M	19.24M
Best val loss	5.7716	5.7668	5.7328	5.7230
Final val loss	5.7634	5.7693	5.7555	5.7247
Final perplexity	318.42	320.32	315.93	306.36
Checkpoint size	122.71 MB	73.44 MB	122.71 MB	73.44 MB
Peak VRAM	10.84 GB	10.69 GB	11.33 GB	11.18 GB
Training time	156.42 sec	144.17 sec	164.86 sec	157.07 sec
Generation latency	10.20 ms/tok	12.64 ms/tok	7.78 ms/tok	15.04 ms/tok
Energy	10.78 Wh	9.87 Wh	11.37 Wh	10.49 Wh

7. Main Observations

Observation	Meaning
Lowest validation loss so far	The combined model produced the strongest language-modeling quality among all tested runs.
Lowest perplexity so far	Perplexity decreased from 318.42 in the baseline to 306.36.
Large parameter reduction	The model used about 40% fewer parameters than the original baseline.
Smaller checkpoint	The checkpoint decreased from 122.71 MB to 73.44 MB.
Energy below baseline	Estimated energy decreased from 10.78 Wh to 10.49 Wh.
Latency drawback	Generation latency increased to 15.04 ms/token, worse than the baseline and SwiGLU-only run.

8. Efficiency Analysis

This experiment is especially important for the Logos goal because it improved quality while reducing the number of parameters and storage requirement. The model became smaller than the baseline, but also achieved better validation loss and perplexity. This means Experiment 5 improved capability per parameter and capability per checkpoint byte.

Efficiency Dimension	Baseline	Experiment 5	Change
Parameter count	32.16M	19.24M	about 40.16% fewer
Checkpoint size	122.71 MB	73.44 MB	about 40.15% smaller
Final validation loss	5.7634	5.7247	lower is better
Final perplexity	318.42	306.36	lower is better
Energy	10.78 Wh	10.49 Wh	slightly lower
Generation latency	10.20 ms/token	15.04 ms/token	worse

9. Scientific Conclusion

Experiment 5 is the best overall run so far for capability-per-parameter and capability-per-storage. The combined Weight Tying + SwiGLU model achieved the lowest validation loss and perplexity while using about 40% fewer parameters than the original baseline. This shows that architectural efficiency improvements can reduce model size without sacrificing quality.

The main tradeoff is inference latency. Even though the model is smaller and more accurate under this fixed training budget, generation latency was worse than the original baseline and much worse than the SwiGLU-only run. Therefore, this version should be considered the strongest quality-and-storage efficient model so far, but not the best latency-optimized model.

10. Limitations

Limitation	Explanation
Fixed short training budget	The run used only 1500 iterations, so longer training could change rankings.
Limited dataset subset	Only 10,000 OpenWebText samples were used.
Latency instability	Generation latency can vary due to GPU state, cache behavior, and runtime noise.
No hyperparameter tuning	Learning rate, dropout, and hidden size were not tuned specifically for this combined model.
No external benchmark yet	Evaluation is mainly based on validation loss, perplexity, and generated samples.

11. Recommended Next Experiment

The next recommended experiment is to test a larger context length by increasing `block_size` from 256 to 512. This will evaluate whether giving the model more context improves validation loss and generation quality, while measuring the cost in VRAM, training speed, latency, and energy.

Next Experiment	Purpose
Larger Context Length, <code>block_size</code> = 512	Measure the quality and efficiency tradeoff of longer context.

12. One-Line Summary

Weight Tying + SwiGLU produced the strongest Logos model so far for validation quality, perplexity, parameter efficiency, and checkpoint efficiency, but it did not improve generation latency.